

Memory Management

Paging

CS232 Operating Systems Fall 2019

Taj Muhammad Khan

Segmentation

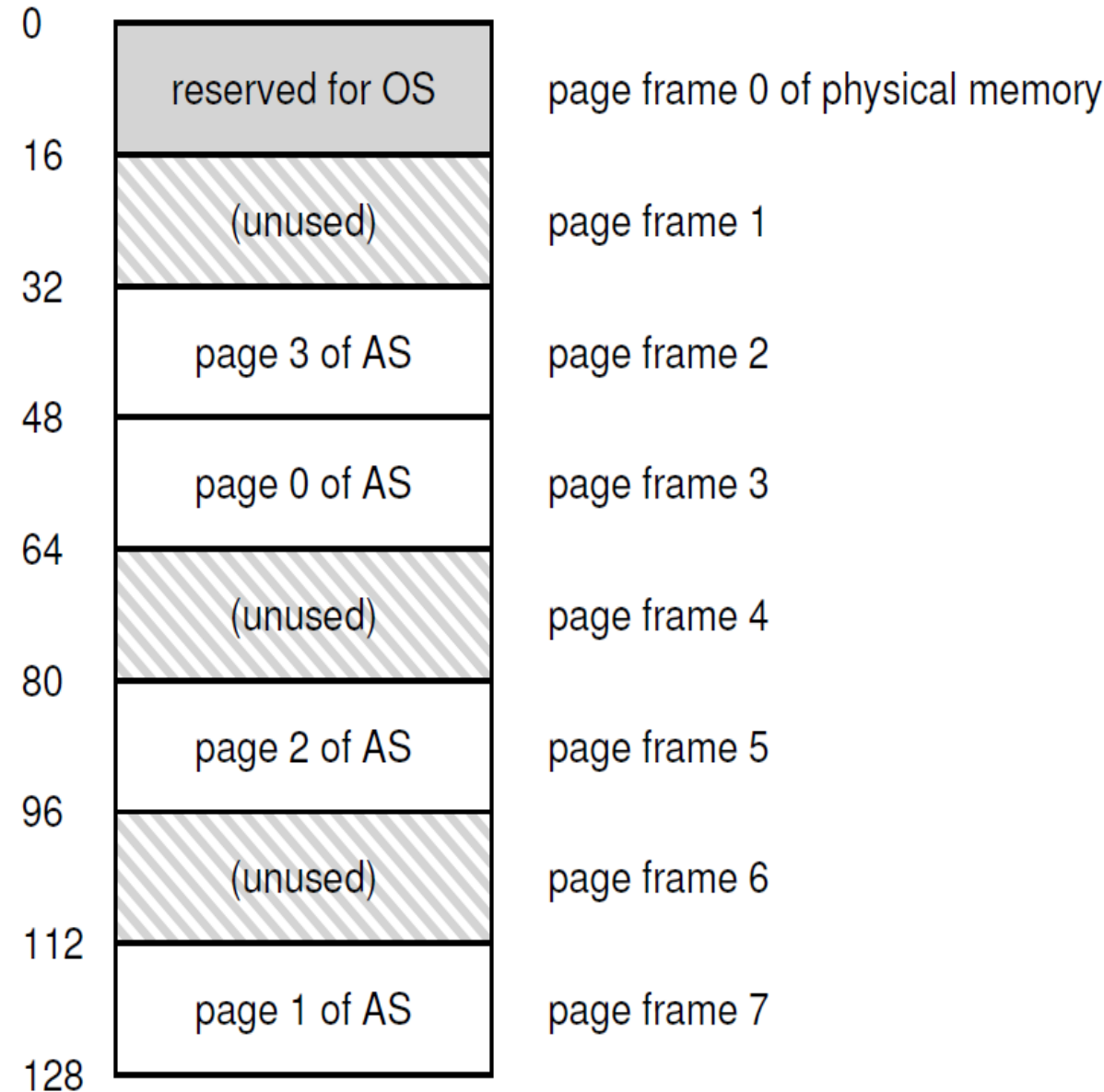
- Segmentation is useful in saving memory.
- Unfortunately it inherently results in external fragmentation
- Mostly because
 - Segments are of unequal sizes
- Another approach divides a process address space in equal sizes

Paging

- Paging divides the virtual address space into fix-sized units called pages.
- The physical memory is correspondingly viewed as an array of fix-sized slots called page frames
- Each page frame (physical) can contain a page (virtual)

Example

- A 64-byte address space
- Divided into pages of 16 bytes each
- Virtual address = page_no. + offset
- Pages can be placed anywhere in physical memory.



Advantages

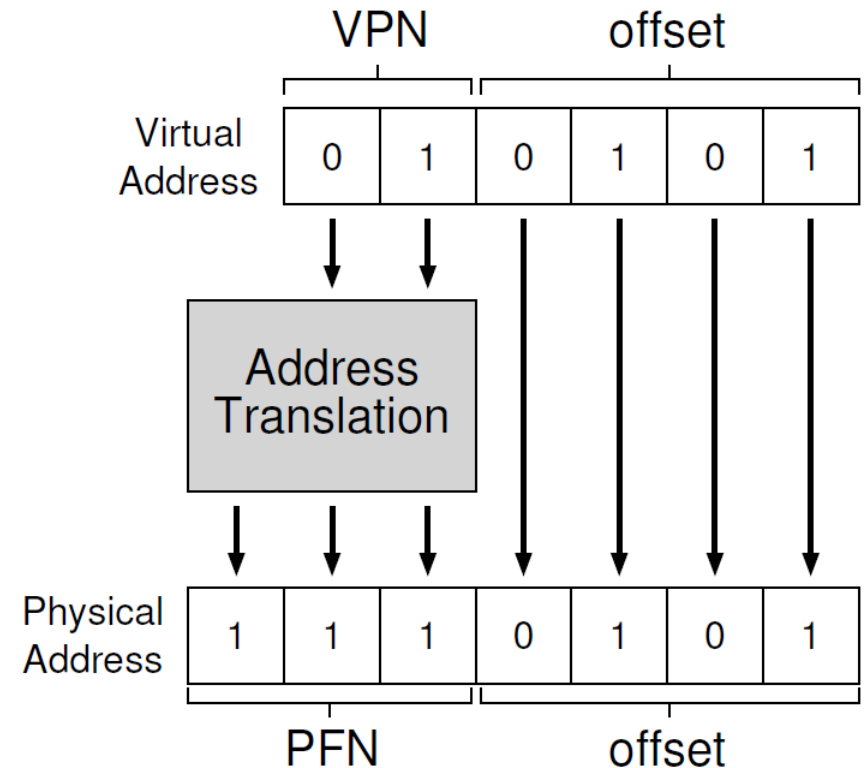
- `page_size == page_frame_size`
- maintenance of *free list* is simple
- doesn't have to worry about stack and heap growing in diff. directions

Address Translation

- The OS needs to store *page tables*
- Page tables store address translations for virtual pages

011
111
101
010

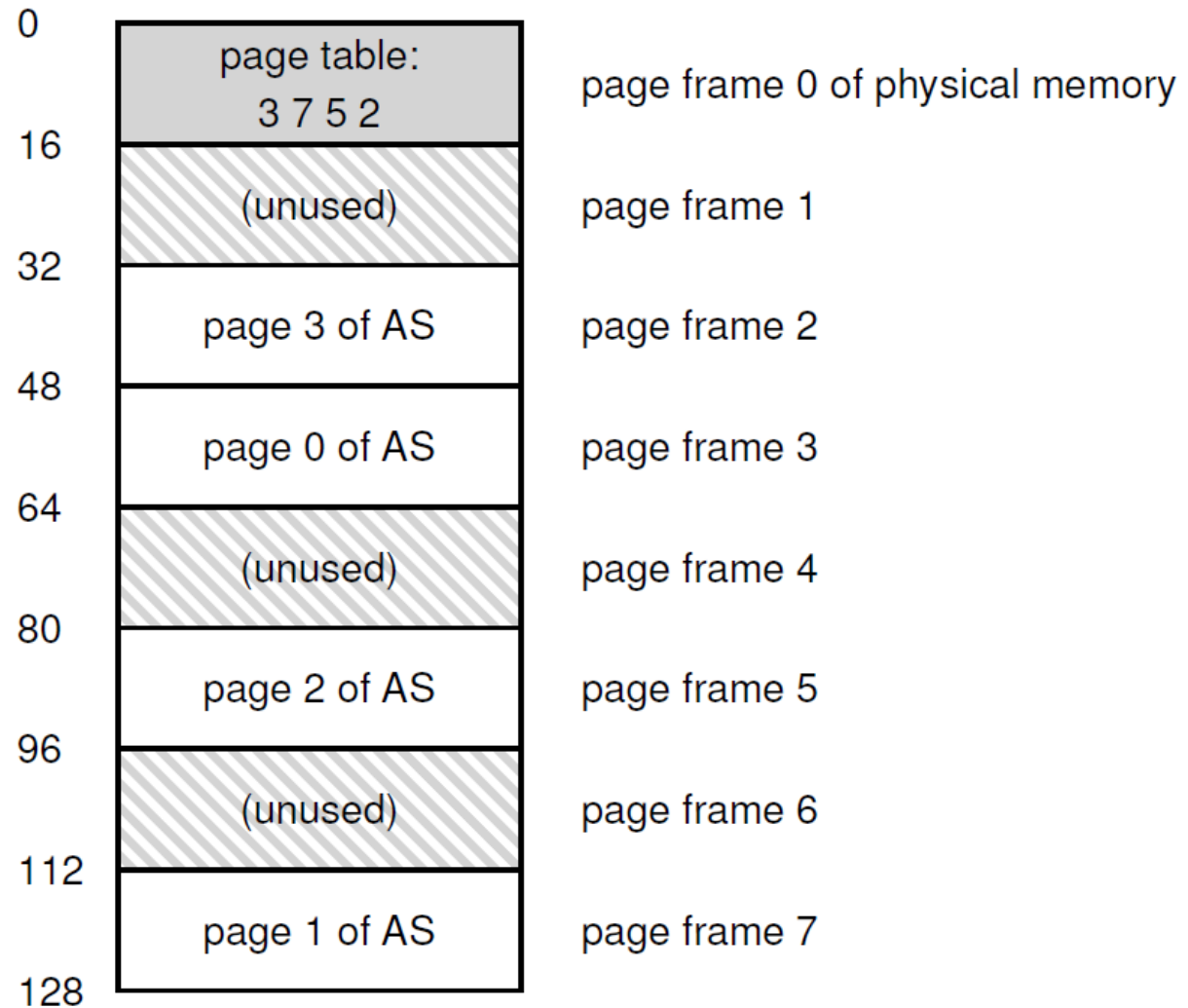
- Pages tables are stored *per-process*



Where are page tables stored?

- What's the size of a page table?
- Imagine a 32-bit address space with 4KB pages
32 bit virtual address = 20 bits for page no. + 12 bits offset
- 20 bits per page no. mean our page table should have 2_{20} page table entries (PTEs) → that's about a million
- If each PTE is 4 bytes, than that's a page table of 4 MB
- Remember a page table is a per-process structure? What if there are 100 processes running?

Where are page tables stored? ... contd.



What's in a Page Table Entry (PTE)?

- Page table stores address translations
- Could be implemented by any data structure
- Simplest form is an array
- The array index is the virtual page number
- The value at that index is the physical page frame number + info

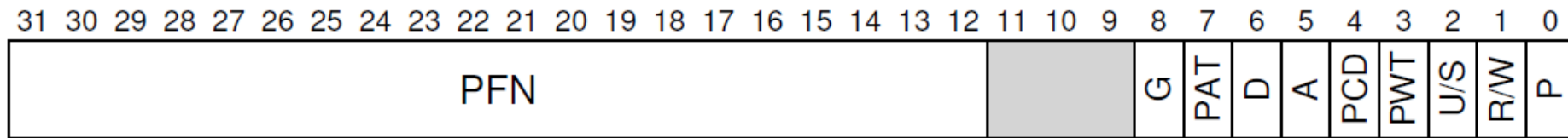


Figure 18.5: An x86 Page Table Entry (PTE)

Is paging slow?

- Where is my page table located in memory?
 - Page Table Base Register

- Address Translation steps:

```
VPN      = (VirtualAddress & VPN_MASK) >> SHIFT
PTEAddr = PageTableBaseRegister + (VPN * sizeof(PTE))
```

- read physical Page Frame Number from PTEAddr and use it to get Physical Address:

```
offset    = VirtualAddress & OFFSET_MASK
PhysAddr = (PFN << SHIFT) | offset
```

- Read data from PhysAddr

Is paging slow? ... contd.

```
1  // Extract the VPN from the virtual address
2  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
3
4  // Form the address of the page-table entry (PTE)
5  PTEAddr = PTBR + (VPN * sizeof(PTE))
6
7  // Fetch the PTE
8  PTE = AccessMemory(PTEAddr)
9
10 // Check if process can access the page
11 if (PTE.Valid == False)
12     RaiseException(SEGMENTATION_FAULT)
13 else if (CanAccess(PTE.ProtectBits) == False)
14     RaiseException(PROTECTION_FAULT)
15 else
16     // Access is OK: form physical address and fetch it
17     offset = VirtualAddress & OFFSET_MASK
18     PhysAddr = (PTE.PFN << PFN_SHIFT) | offset
19     Register = AccessMemory(PhysAddr)
```

Figure 18.6: Accessing Memory With Paging

Is paging slow? ... contd.2

- For each memory access we access RAM twice:
 - Once to read PFN from the page table
 - Second to read the actual data

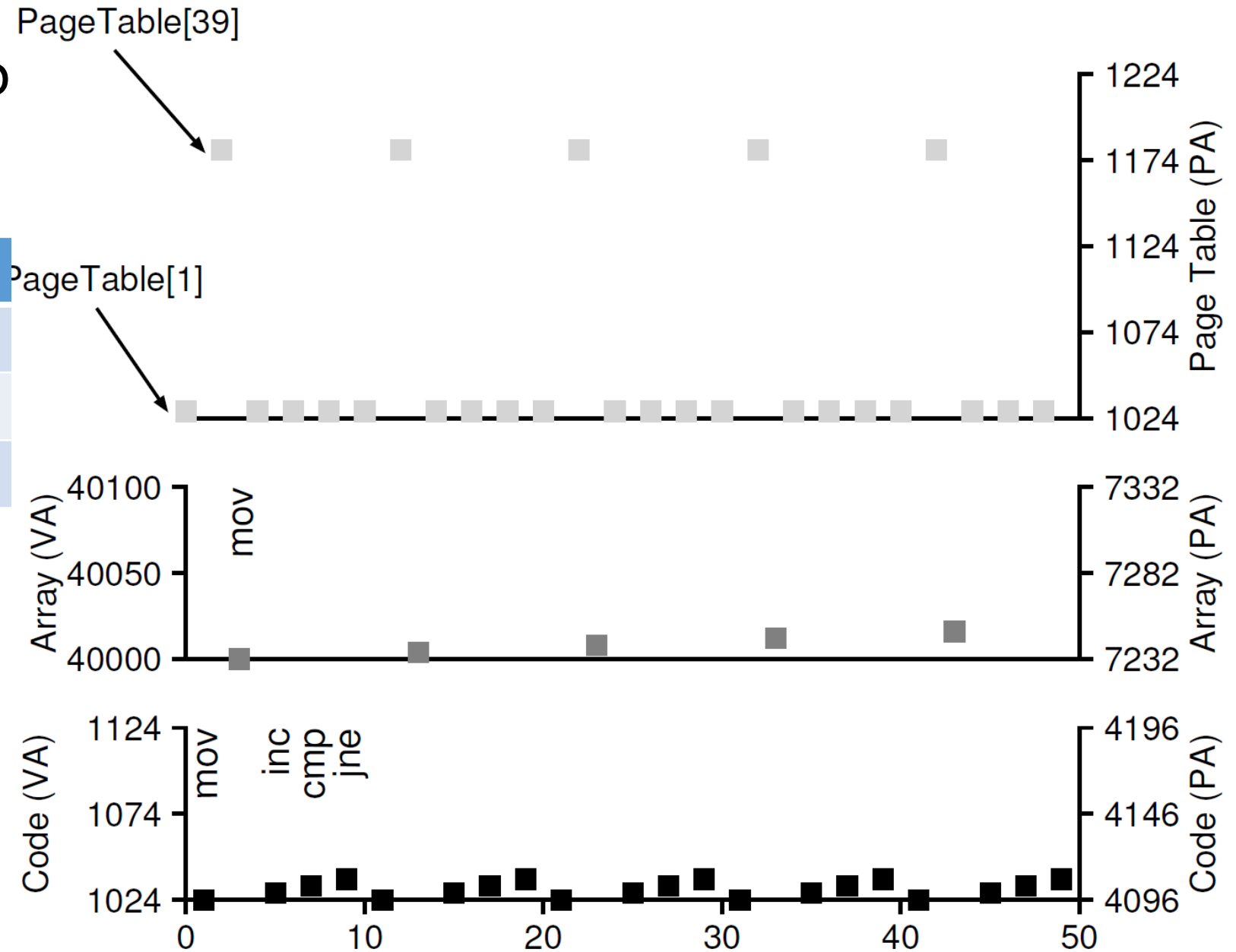
Example:

```
int array[1000];  
...  
for (i = 0; i < 1000; i++)  
    array[i] = 0;
```

```
1024 movl $0x0, (%edi,%eax,4)  
1028 incl %eax  
1032 cmpl $0x03e8,%eax  
1036 jne 0x1024
```

Is paging slow?

	V.A	VPN	P.A	PFN
P.Table			1024	1
Code	1024	1	4096	4
Array	40000	39...42	7232	7...10



Paging: analysis

- Pros
 - Solves external fragmentation
- Cons
 - Slows down the execution
 - Wastes memory